



Leveraging deep edge intelligence for real-time respiratory disease detection



Tahiya Tasneem Oishee, Jareen Anjom, Uzma Mohammed, Md. Ishan Arefin Hossain *

Department of Electrical and Computer Engineering, North South University, Bashundhara R/A 1229, Dhaka, Bangladesh

ARTICLE INFO

Article history:

Received 4 September 2024

Accepted 3 January 2025

Available online 8 January 2025

Keywords:

Respiratory disease

ICBHI

Deep learning

Early diagnosis

Edge intelligence

ABSTRACT

Detecting respiratory diseases such as COPD, bronchiolitis, URTI, and pneumonia is crucial for early medical intervention. This study utilizes the ICBHI dataset to train and evaluate deep learning architectures such as CNN-GRU, VGGish, YAMNet, CNN-LSTM, and basic CNN to automate this process. After a detailed analysis of the performance of these models, the CNN-LSTM model achieved an impressive accuracy and F1 score of 96% each. The model is also considerably lightweight, as its weights are further pruned and then quantized using TensorFlow Lite (TFLite), with the model being optimized at a significantly small size of 0.38 MB with only a loss of about 1% in performance. Subsequently, this was deployed to the smartphone application RespiScan. The application uses the prediction capabilities of the disease detection model on patients' audio recordings. By providing a portable, cost-effective, and efficient, lightweight solution for respiratory health monitoring, this work contributes significantly to timely disease detection. It promotes proactive health management, thereby reducing the burden on healthcare systems. This work can be further validated in real-world conditions, such as for initial preliminary auscultation purposes, to ensure the proposed work's efficacy across different environmental settings.

© 2025 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co., Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Respiratory diseases are major global health concerns, causing significant death and disability. COPD affects around 200 million people, leading to 3.2 million deaths annually, making it the third-leading cause of death worldwide. Asthma, the most common chronic childhood disease, impacts over 350 million people, while pneumonia claims more than 2.4 million lives each year.¹ Given the significant global burden of COPD, asthma, and pneumonia, similar to other noncommunicable diseases like heart disease and diabetes, early and accurate diagnosis is crucial for prevention, timely intervention, and improved patient outcomes.

Traditional diagnostic methods for respiratory illnesses can be categorized into two main approaches: imaging-based and lung function testing. Chest X-rays and computed tomography (CT) scans are commonly used imaging techniques, but they can be expensive. In 2022, the cost of a routine chest X-ray examination was approximately USD 226.44.² Moreover, the imaging techniques can expose patients to ionizing radiation. Although a single imaging session may pose minimal risk, repeated imaging for a

patient can lead to significant radiation exposure. Physicians commonly rely on manual auscultation to assess lung sounds, a crucial method for diagnosing diseases.³ Skilled expertise is essential due to challenges like instrument calibration issues and background noise interference such as heartbeats and coughing.⁴ These challenges are more pronounced in underserved areas and during crises, such as pandemics (e.g., COVID-19), where there is a shortage of experienced medical staff. Automated analysis of respiratory sounds can address these issues and facilitate telemedicine, allowing less experienced personnel, such as community health workers, to monitor patients remotely. The automation of respiratory diagnosis was explored in the study.⁵ Their implementation is further discussed in Section 2.

The integration of deep learning models in disease diagnosis offers significant advancements in medical technology but raises several ethical considerations, particularly regarding data privacy, bias, and their impact on patient care. Medical data used for training deep learning models often contains highly sensitive information. It is important that healthcare organizations comply with regulatory standards like the Health Insurance Portability and Accountability Act (HIPAA) in the U.S. and the General Data Protection Regulation (GDPR) in Europe to protect patient data privacy and security.⁶ Another ethical concern in using deep learning mod-

* Corresponding author.

E-mail address: ishan.hossain@northsouth.edu (M.I.A. Hossain).

els for disease diagnosis is the potential for bias in the models. If the training data used to develop these models does not represent a diverse patient population, the existing healthcare disparities will remain unaddressed. Therefore, it is essential that the data used to train deep learning models include a wide range of ethnicities, ages, genders, and socioeconomic backgrounds to avoid misdiagnosis and ensure inclusive healthcare outcomes. The use of AI in healthcare may also disrupt the traditional physician-patient relationship, as patients might feel disempowered by AI-based diagnoses, potentially reducing trust in the healthcare system. It should be emphasized that AI integration in medical diagnosis, such as in medical imaging, enhances diagnostic accuracy, efficiency, and standardization, supporting healthcare professionals in disease detection, treatment planning, and workflow management while complementing human expertise.⁷

Developing nations face significant challenges within their healthcare systems, particularly due to a shortage of trained personnel, especially in remote areas distant from major cities. For instance, in Colombia, there are only 1.5 medical doctors per 1000 residents, and merely 0.4 specialist doctors per 1000 residents, highlighting a critical issue. This deficiency not only exacerbates health risks such as respiratory diseases like COPD, particularly in areas with low air quality but also underscores the inadequate health infrastructure, lacking essential facilities like laboratories and properly equipped consulting rooms. These circumstances underscore the pressing need for more solutions to support healthcare professionals in their demanding roles.

By leveraging deep learning for respiratory sound analysis, we aim to develop a tool that can empower healthcare professionals in developing nations to deliver more accurate and accessible diagnoses, particularly in underserved and remote areas. Recent advancements in deep learning have shown promise in medical applications, particularly in respiratory disease diagnosis. Deep learning models trained on audio recordings of breath sounds, such as wheezing and crackles, can identify characteristic patterns associated with conditions like asthma, COPD, pneumonia, and lung fibrosis. Wheezing typically appears as a high-pitched sound ranging from 100 to 2500 Hz,^{8,9} while crackles, indicative of various respiratory illnesses, vary in frequency and duration.^{10,11}

This study proposes a deep learning framework for respiratory disease diagnosis using audio recordings. We address the aforementioned challenges through the following contributions:

- **Multi-Feature Extraction:** We extract Mel-frequency cepstral coefficients (MFCCs), chroma spectrograms (Chroma-stfts), and Mel-spectrograms (M-Spectrograms) from audio recordings, capturing complementary information about the respiratory sounds.
- **Data Augmentation Techniques:** We employ data augmentation techniques to artificially expand the training dataset, improving model generalizability and robustness.
- **Lightweight Model Architecture:** We design a lightweight Convolutional Neural Network (CNN) architecture that is further pruned and quantized to make it suitable for mobile device deployment.

In this study, we propose a deep learning-based solution for the early detection of respiratory diseases from audio recordings of lung auscultations. Our approach utilizes a CNN-LSTM model, which has been optimized for deployment on edge devices such as smartphones. By quantizing the model, we ensure a lightweight and efficient model is achieved without significantly compromising its accuracy. The model is integrated into a mobile application, RespiScan, which allows us to perform lung auscultation using a smartphone, offering a cost-effective and portable diagnostic tool.

This solution aims to improve respiratory disease detection, enhancing access to timely and accurate medical interventions.

This paper is organized into different sections as follows: Section 2 reports the existing related literature and their limitations, Section 3 describes the detailed methodology of the system, Section 4 evaluates the system and compares other architectures. Finally, the paper is concluded in Section 5.

2. Related works

There are quite a few recent related works that focus on simplifying the process of detecting respiratory diseases through sound.

In the paper by Arpan Srivastava et al.,¹² Convolutional Neural Network based deep learning methodologies have been used on the ICBHI dataset to provide a detailed analysis for Chronic Obstructive Pulmonary diseases. The system also interprets the severity of the identified disease, such as mild, moderate, or acute and. They have utilized feature extraction techniques such as Mel-Frequency Cepstral Coefficients (mfcc), melspectrogram (Mel-Spectrogram), Chromagram calculated from the waveform/power spectrogram (chroma_stft), ConstantQ Chromagram (chroma_cqt) and chroma_cens (Chroma Energy Normalized Variant (CENS)) and applied augmentation techniques such as loudness, mask, shift, and speed. However, their proposed model is able to classify the diseases into only two categories: COPD and non-COPD. The model received a classification accuracy of 93%. As the model is efficient with good performance, a device could have been implemented to aid users accessibility to the system.

Yoonjoo Kim et al.¹³ developed an automated classification of breathing sounds using deep CNN that categorizes respiratory sounds of a custom dataset recorded in a clinical setting. This system received an overall accuracy of 85.7% for detecting crackles, wheezes, and rhonchi. Their model uses pre-trained VGG16 model as the feature extractor and a 1-layer CNN as the classifier. However, it also lacks an accessible device for handling cough detection.

Kumari Nidhi Lal¹⁴ has proposed a combined VGGish BiGRU model to classify healthy and COPD respiratory sounds. He has applied denoising and Mel Spectrogram for preprocessing of audio data. His proposed model achieved macro average of 94% which is the mean of average of all classes.

In the study by Victor Basu et al.,¹⁵ a five layer GRU(Gated Recurrent Unit) based neural network have been deployed for classification of respiratory diseases from the ICBHI dataset. Data augmentation have been performed to address the class imbalance issue. MFCC(Mel-frequency Cepstral Coefficients) technique has been used for feature extraction. The model has achieved an accuracy of $95.67 \pm 0.77 \%$. However, no application has been implemented for easy detection of respiratory diseases from lung auscultation.

Naoki Asatoni et al.¹⁶ have employed an Improved CRNN(Convolutional Recurrent Neural Network) for the respiratory sounds classification by generated images. The images generated step was done using several feature extraction techniques such as Short-Time Fourier Transform, Constant-q Transform, and Continuous Wavelet Transform. The resulting images of each feature extraction methods were concatenated and passed into their proposed model. The improved CRNN is an original CRNN model with Bi-LSTM (Bidirectional LSTM) and 1D Global Average Pooling (GAP) layers. Their model have achieved sensitivity = 0.64, specificity = 0.83, average score = 0.74, and harmonic score = 0.72.

In the study by Lei Xu et al.,¹⁷ an adventitious respiratory sound classification network (ARSC-Net) with a combination of residual block with channel-spatial attention have been proposed. Several preprocessing techniques have been performed and MFCC and Mel-spectrogram methods have been utilized for feature extrac-

tion. Their model performed better than other state-of-the-art models with sensitivity, specificity and scores of 46.38%, 67.13% and 56.76% respectively for the four-class sound classification (normal, crackles, wheezes and both).

Lam Pham et al.¹⁸ have proposed a CNN-MOE architecture which is a C-DNN architecture incorporated with a mixture-of-experts (MoE) technique into the DNN part of the network. They have carried out both respiratory anomaly classification (Crackle, Wheeze, Both, and Normal) and respiratory disease prediction Healthy, Chronic Disease (i.e. COPD, Bronchiectasis and Asthma) and Non-Chronic Disease (i.e. upper and lower respiratory tract infection, Pneumonia, and Bronchiolitis). They have also analyzed the impacts of different feature extraction techniques, data augmentation, and train-test split on the performance of the model. They further performed knowledge distillation, which resulted in a slight depreciation in performance. However, providing the model sizes before and after knowledge distillation would have offered a clearer comparison. For train-test splits of 5 folds, their model has achieved performance metrics of sensitivity = 0.98, specificity = 0.86, and score = 0.92. With their impressive performance metrics, they could have implemented it in an embedded device.

In a study by M. Fraiwan, et al.,¹⁹ a combined dataset of 103 patients obtained from locally recorded stethoscope lung sounds in Jordan, and 110 patients from the ICBHI dataset was trained upon with several preprocessing steps (wavelet smoothing, displacement artifact removal, z-score normalization) to ensure smooth and less noisy signals. The model architecture has CNNs and bidirectional long short-term memory (BiLSTM) units. This system achieved an overall accuracy of 99.62% and a precision of 98.85% to classify pulmonary diseases (asthma, pneumonia, BRON, COPD, and HF). Implementing the model on an edge device could make it usable in clinical diagnosis. Moreover, a BiLSTM model involves more parameters compared to a LSTM model, making it computationally more expensive.

In the study by Viswam Nathan et al.,²⁰ they have deployed a YAMNet model with a binary classifier network to perform recognition of asthma and COPD conditions from speech audio samples. The subject-level performance metrics for speech recordings achieved accuracy, precision, recall, and F1-score of 81.0%, 88.3%, 88.8%, and 88.3% respectively.

The automation of respiratory diagnosis was explored in the study by Daniel B. Chamberlain et al.⁵ They proposed a mobile application for the real-time detection of COPD and asthma. They

employed a two-stage logistic regression model that effectively distinguished asthma or COPD patients from the general population, achieving an ROC curve with an AUC of 0.95. Furthermore, their model was able to differentiate between asthma and COPD patients, yielding an ROC curve with an AUC of 0.97. In contrast, our proposed mobile application addresses the detection of five respiratory conditions with high accuracy, achieving ROC curve AUC values of 0.99, 1.00, 0.99, 0.99, and 0.98 for Bronchiolitis, COPD, Healthy, Pneumonia, and URTI classes, respectively.

A critical analysis of the works has been presented in Table 1.

We deployed models similar to the ones proposed in studies,^{12,13,15, 20} and analysed their respective performances in Section 4. Among these, our proposed CNN-LSTM model demonstrated superior performance. The CNN-LSTM architecture was selected for its ability to leverage the feature extraction strengths of CNN alongside the sequence learning capabilities of LSTM networks. LSTM is preferred over traditional Recurrent Neural Networks (RNNs) as it effectively addresses the vanishing and exploding gradient problems, which hinder RNNs from accurately capturing long-term dependencies.¹⁴ While Bidirectional LSTM (BiLSTM) offers the advantage of processing data in both forward and backward directions, thereby improving model performance by capturing more context, this comes at the cost of increased computational complexity. Given the trade-offs, LSTM was proved to be more suitable for the objectives of this study, as shown by the performance comparisons in Section 4. The configurations and architectures of the deployed models are detailed in Section 3.5.

To the best of our knowledge, there is no work that establishes an automated system of detecting respiratory diseases using an accessible device, such as a smartphone.

3. Methodology

The methodology of this paper is described in this section, and is also shown in Fig. 1.

3.1. Dataset

The Respiratory Sound database - ICBHI 2017 Challenge²¹ is used to perform respiratory disease classification. The database consists of a total of 5.5 h of recordings containing 6898 respiratory cycles, of which 1864 contain crackles, 886 contain wheezes, and 506 contain both crackles and wheezes, in 920 annotated audio

Table 1
Comparison analysis between prediction models.

Ref	Author	Model	Features	Metrics	Limitations
12	Arpan Srivastav et al.	CNN-based	4 Conv2D Layers with Max2DPooling	Accuracy = 93%	Average metrics
13	Yoonjoo Kim et al.	VGG-CNN	VGG16 feature extractor, CNN classifier	Accuracy = 85.7%	Average metrics
14	Kumari Nidhi Lal	VGGish-BiGRU	VGGish for transfer learning, Backward and Forward GRU as classifier	Precision = 95%, Recall = 94%, F1-score = 94%	No mobile applications
15	Victor Basu et al.	GRU-based	5 layers	Accuracy = 95.67%±0.77%	No mobile applications
16	Naoki Asatoni et al.	CRNN	CRNN with Bi-LSTM and 1D GAP layers	sensitivity = 0.64, specificity = 0.83, average score = 0.74, and harmonic score = 0.72	Average metrics
17	Lei Xu et al.	ARSC-Net	Residual block with channel-spatial attention	sensitivity = 46.38%, specificity = 67.13% and scores = 56.76%	Average metrics
18	Lam Pham et al.	CNN-MOE	C-DNN with MOE techniques	sensitivity = 0.98, specificity = 0.86, and score = 0.92	Average metrics; no mobile applications
19	M. Fraiwan et al.	CNN based	CNN with bidirectional LSTM units	Accuracy = 99.62%, Precision = 98.55%	No mobile applications
20	Viswam Nathan et al.	YAMNet	YAMNet for transfer learning, dense network with ReLU activation as classifier	Accuracy = 81.0%, Precision = 88.3%, Recall = 88.8%, F1-Score = 88.3%	Average metrics, No mobile applications
5	Daniel B. Chamberlain et al.	Logistic Regression	Two-stage logistic regression	AUC = 0.97, AUC = 0.95	No deep learning models employed Only 2 disease classes

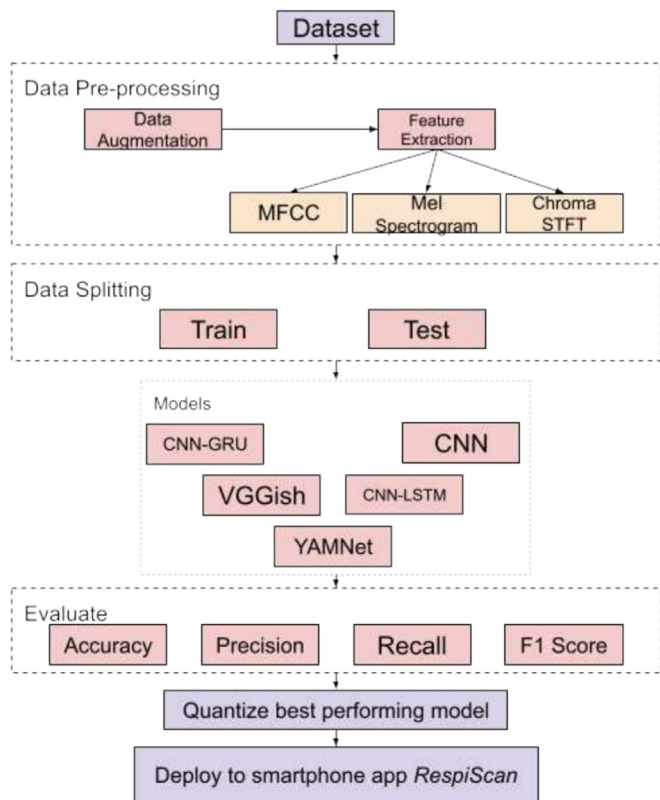


Fig. 1. System architecture.

samples from 126 subjects. The respiratory sounds in the dataset are of eight different categories: Healthy, COPD(Chronic obstructive pulmonary disease), URTI(upper respiratory tract infection), Bronchiectasis, Pneumonia, Bronchiolitis, Asthma, LRTI(Lower respiratory tract infection). Each of the patients' data and demographic information is also provided. The disease classification is chosen to identify the different diseases in the respiratory sounds.

3.2. Exploratory data analysis

Upon analysing the data, it was observed that the class COPD has the highest number of samples while Asthma and LRTI classes

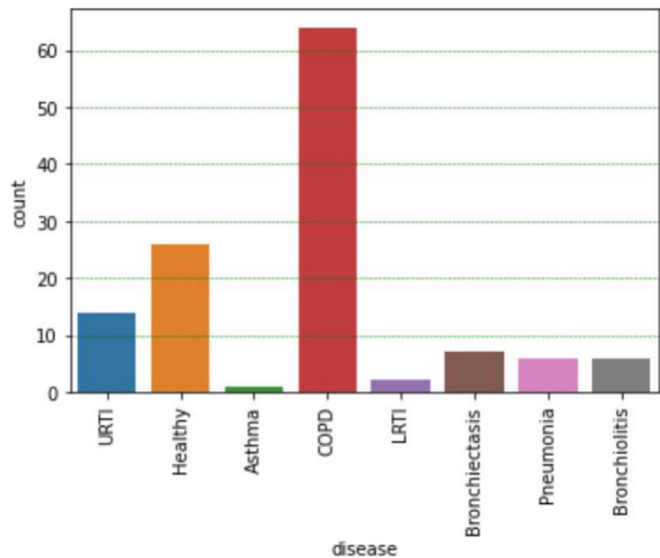


Fig. 2. Class sample imbalance.

have the lowest. Other classes also had a significant difference in samples compared to COPD as seen in Fig. 2. This caused a huge imbalance in the data. Class imbalance can have adverse effects on the performance of deep learning models, especially for classification problems. Models may become biased towards the majority class, overlooking the minority classes, impacting their effectiveness and reliability. This can lead to misdiagnosis of less common diseases.

3.3. Data pre-processing

The ICBHI respiratory sound database comes with annotation files that consists of four columns: beginning of respiratory cycle (s), end of respiratory cycle(s), presence/absence of crackles (presence = 1, absence = 0), and presence/absence of wheezes (presence = 1, absence = 0). Using the timestamps of the respiratory cycles, the audio samples were segmented into shorter audio signals, each consisting of one respiratory cycle. Padding was applied to ensure that all the audio samples are of uniform lengths. This is done by adding extra zeros to the shorter samples to match the length of the longest sample. This step was important for maintaining consistent input dimensions for the deep learning models, which expect fixed-length inputs.

To counter the class imbalance issues in the dataset, several preprocessing steps were performed. One crucial step was the combination of the Bronchiectasis and Bronchiolitis classes into a single class, labeled as Bronchiolitis. This was necessary due to the individually low sample counts for these two classes. This was done because both conditions share similar symptoms and are both bronchial diseases. Furthermore, the combined class was a necessity due to limitations of data availability and to create a more balanced dataset for model training. However, this combination may affect the models' interpretability and clinical relevance. Although the models' predictions may become simpler, there is a chance of losing specificity. Merging the two conditions may affect the models' ability to differentiate between the two conditions when precise treatment is critical. Asthma and LRTI classes were removed as they have significantly less samples compared to the other classes. Including these classes could pose an over-fitting problem of the models, resulting in biased and unreliable model classifications. Moreover, including them could compromise the overall interpretability of the models for the other well represented classes.

Various data augmentation techniques such as, adding noise, shifting, stretching, and pitch shifting have been applied to the classes URTI and Pneumonia to increase samples. $add_noise(data, x)$ function creates a noisy version of the original signal by generating a random noise array, scaling the noise by a scale factor of x and adding the scaled noise to the original audio $data$. Adding noise to the audio allows the model to become more robust to variations in the data. A scale factor of 0.001 was chosen. $shift(data, x)$ uses the $roll(data, x)$ function to shift the audio sample by x positions. 1600 was given as the position parameter. Shifting the audio makes the model less sensitive to the exact timing of features. The librosa library function $time_stretch(data, rate)$ stretches or compresses the audio by a given factor of $rate$ while keeping the pitch unchanged. The audio samples were stretched and compressed by scales of 1.2 and 0.8 respectively. Time stretching introduces variation in the speed of the audio, making the model more robust to changes in audio speed. Lastly, the librosa library function $pitch_shift(data, sr = 220250, n_steps = rate)$ was used to shift the audio pitch by $rate$ steps. Pitch shifting makes the model more sensitive to changes in pitch. These augmentations allow diversification of the training audio data and improvement of the generalization of the models. The data distribution after resampling can be seen in Fig. 3.

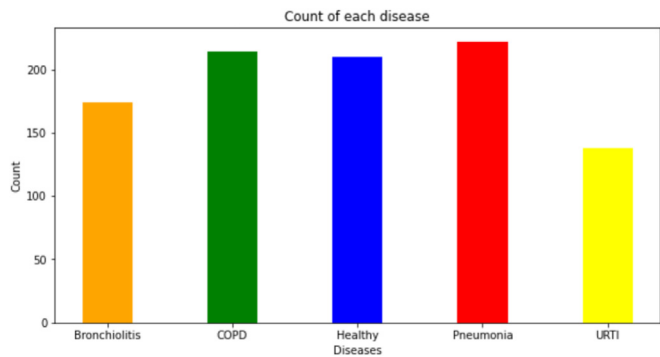


Fig. 3. Resampled class distribution.

Alongside the audio recordings in the ICBHI respiratory sound database, it also included annotation files containing crucial details about each respiratory cycle within an audio file. These annotation files included information such as: the beginning and end times of the respiratory cycle, and indicators for the presence or absence of crackles and wheezes (represented by 1 and 0, respectively). To prepare this data for effective model training, a series of preprocessing steps were implemented.

Firstly, patient information, including IDs and corresponding diseases, was loaded from a CSV file. Simultaneously, separate text files with tab delimiters, located within a specific directory structure, provided annotations for each audio file. The information from these annotation files were extracted and organized into a pandas DataFrame. To ensure efficient merging with patient data later, patient IDs were converted from text format to integers.

After extracting features from the annotations, they were merged with the patient data using the common 'pid' column, creating a single DataFrame that combined patient information and respiratory cycle characteristics. To counter the class imbalance issues in the dataset, several preprocessing steps were performed. Bronchiectasis and Bronchiolitis classes are combined to one class (Bronchiolitis) due to each of them having less samples. Asthma and LRTI classes are removed as they have significantly less samples compared to the other classes.

A custom function named `getPureSample` was defined to extract specific segments from the raw audio files. This function utilized the start and end times of respiratory cycles, along with the sampling rate of the audio data, to precisely extract the relevant portions.

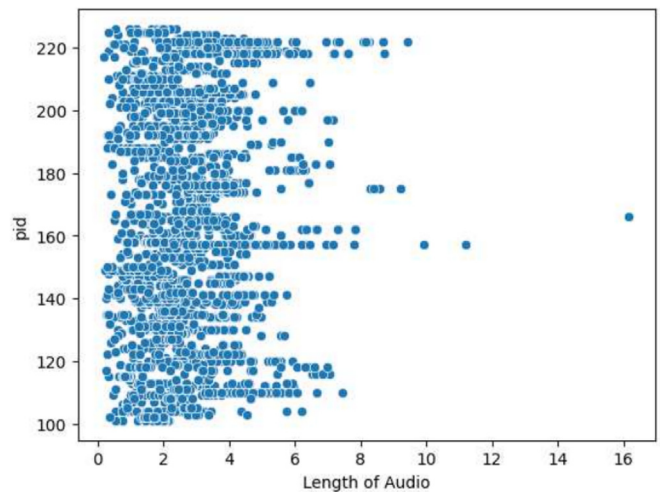


Fig. 4. Analysing scatterplot to find optimal length of audio.

With the goal of achieving uniformity in the data, a standardization step was implemented for the audio segments. Visualizations such as scatterplots (Fig. 4) are employed to determine an optimal fixed length that could be applied consistently to all segments. For segments with a length falling below the chosen standard, zero padding was applied. This technique essentially extended the shorter segments with additional silence (zeros) to match the desired length.

The final step involved creating separate WAV files containing the preprocessed audio segments. The data underwent iteration using pandas' `iterrows` method. During this iteration, respiratory cycle segments were extracted from each audio file and saved as individual WAV files in a designated directory. A unique filename was assigned to each segment, considering the original filename of the audio file and incorporating an index to manage situations where a single audio file encompassed multiple respiratory cycles.

3.4. Feature extraction

For the classification of respiratory sounds, feature extractions play a crucial role. Raw audio data contains many components that are not useful for classification. Feature extraction allows to isolate the most relevant audio information, such as frequencies, cepstral features, harmonic features, and spectral features, which are essential for distinguishing between various audio classes. Three feature extraction techniques have been utilized in this study: Mel-frequency cepstral coefficients (MFCCs), Mel Spectrogram, and Chroma Short Time Fourier Transform (Chroma STFT). These techniques have been employed by simply using built-in functions (`mfcc`, `chroma_stft`, and `melspectrogram`) from Librosa library. Fig. 5 below shows the spectrogram plot for an individual.wav file.

3.4.1. Mel-frequency cepstral coefficients (MFCCs)

MFCC is one of the most widely used feature extraction technique for audio signal analysis. It describes the overall shape of the spectral envelope of a small set of features of an audio signal. MFCC can be calculated following the consecutive processes: pre-emphasis, signal framing and windowing, computing power spectrum, applying a Mel filter bank to the obtained power spectra, finding the logarithmic values of all filter banks, and applying the Discrete Cosine Transforms (DCT). The methods of MFCC feature extraction have been derived from the work of.²²

- Pre-Emphasis: Pre-emphasis the first step to finding MFCC in which the suppressed frequencies of the signals are amplified. This is done by applying a filter with a setting of 1. This boosts the signals of high frequencies while leaving the lower frequencies relatively unchanged.²³
- Signal Framing and Windowing: Audio signals change over time; therefore, they are splitted into frames where the signals are relatively more stable. A window is applied on each frame to minimize the sharp edges at the beginning and ending of each

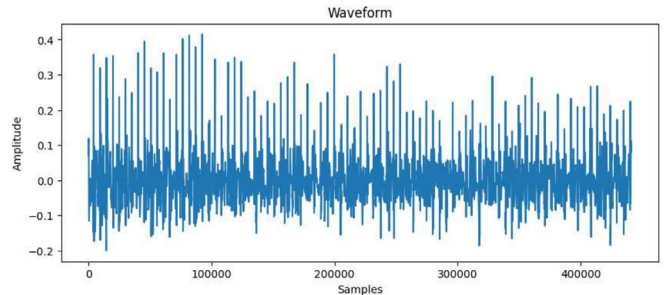


Fig. 5. Spectrogram of an audio sample.

frame. Hanning and Hamming windows are applied for the windowing process.²⁴ Windowing helps to analyze the important features in each frame.

- **Power Spectrum:** Power spectrum shows the distribution of energy of the audio signal across the different frequencies.²⁵ Discrete Fourier Transform (DFT) is used to compute the power spectrum. It does so by breaking down each audio signal into its component frequency and summing the contribution of each frequency.
- **Mel Filter Bank:** The Mel band-pass filter is a collection of filters made based on the perception of pitch. The conventional Mel filter bank consists of a series of triangular filters that captures the pitch information.²⁶
- **Discrete Cosine Transforms (DCT):** A Discrete Cosine Transform (DCT) represents a finite set of data points as the summation of cosine functions oscillating at various frequencies. In the process of Mel-Frequency Cepstral Coefficients (MFCC), the DCT is employed on the Mel filter bank to extract the most important features.²⁷

3.4.2. Mel spectrogram

Mel Spectrogram shows how the energy of different pitches changes over time. It consists of Short Term Fourier Transform (STFT) that is passed through a Mel-frequency filter bank.²⁸ STFT breaks down a signal into its frequency components over short intervals. The following is the equation for computing STFT:

$$S(f, t) = \sum_{n=0}^{N-1} w_n x_t(n) e^{-j2\pi(k/N)n} \quad (1)$$

where, w_n is the Hamming window function, x_t is the speech signal, $k = 1, 2, \dots, N/2 + 1$ is the frequency index, and N is the number of samples in the frame. The Mel filter bank is then applied to the magnitude of STFT as shown in the equation below:

$$S_{mel}(k, t) = \sum_{l=0}^{L-1} m_k(l) |S(l, t)|^2 \quad (2)$$

where L is the frequency component number and $m_k(l)$ is the k filter of the mel filter bank.

3.4.3. Chroma Short Time Fourier Transform (Chroma STFT)

Chroma STFT is used to extract chroma features from audio samples. It captures the pitch class profiles of a signal. An STFT is computed to calculate the chroma features. The chroma vector is computed by mapping the magnitude of STFT to chroma values. These features are used to differentiate between different pitch class profiles of audio signals.²⁹

$$\text{Chroma}(c, m) = \sum_{k \in K_c} |X(m, k)| \quad (3)$$

where c is the chroma values, K_c is the set of frequencies, and $X(m, k)$ is the STFT at time frame m .

3.5. Training phase

The MFCC, Mel Spectrogram, and Chroma STFT features of the audio recordings extracted are mapped to their corresponding class of respiratory diseases. The dataset is then split in 80:20 ratio, where 80% of the data the training set and the rest 20% is the testing data. The spectrogram features are fed to the classifiers for categorization of respiratory diseases. According to the analysis of related methods used Section 2, we consider The following models to evaluate and analyze their performances on the pre-processed data.

3.5.1. CNN-GRU

Gated Recurrent Unit (GRU)³⁰ is a variant of Recurrent Neural Network (RNN) architecture developed to handle sequential data

and capture temporal dependencies. It addresses the problem of vanishing and exploding gradients encountered during training of traditional RNNs.

The CNN-GRU model employed in this paper has an input layer of shape (1, 52), followed by two convolutional layers. These two layers include ReLU activation, followed by max pooling and batch normalization.

Next, the model branches into three parallel sequences of GRU layers. Each branch includes GRU layers with varying units (32, 64, and 128), all with *return_sequences = True* and *go_backwards = True*, to capture temporal dependencies. The outputs of these branches are then combined using an addition layer. The combined output is processed further by two sets of GRU layers with the same settings as before.

The final layers consist of dense fully connected layers with LeakyReLU activation. The model concludes with additional dense layers and a softmax activation layer for classification. This architecture allows the model to effectively extract features and capture temporal dynamics for audio classification tasks.

3.5.2. VGGish

VGGish³¹ is a variant of the VGG (Visual Geometry Group) architecture,³² developed by Google for audio classification tasks. It was trained on AudioSet,³³ a dataset containing over 2 million human-labeled 10-s YouTube video soundtracks encompassing more than 600 audio event classes.

Instead of taking raw audio inputs, VGGish processes log mel spectrograms, ideally sized at 96x64. Its architecture closely mirrors that of the original VGG model. It features a series of convolutional layers with 3x3 filters, organized in blocks with increasing filter counts. Each block of convolutional layers is followed by max pooling layers with 2x2 filters. The model concludes with three fully connected layers, two of which typically containing 4096 neurons, and utilizing ReLU activation functions. The final layer employs a dense layer and a softmax activation function for classification tasks.

Fig. 6 illustrates the VGGish model architecture. Purple rectangles represent 2D convolutional layers, yellow rectangles indicate max pooling layers, and orange rectangles denote fully connected layers. The numbers inside the purple rectangles indicate the number of filters, while the numbers inside the orange rectangles specify the number of neurons.

3.5.3. YAMNet

YAMNet (Yet Another Multi-scale Convolutional Neural Network)³⁴ is a deep learning model developed by Google that utilizes MobileNetV1³⁵ as its base network. YAMNet processes log mel spectrograms as input, typically represented as a 2D tensor.

The core components of the YAMNet architecture include standard convolution and depthwise separable convolutions (DSC). In the DSC component, a depthwise convolution is applied using a single filter for each input channel, followed by a pointwise convolution that uses a 1x1 convolution to combine the outputs of the depthwise convolution.

After the convolutional layers, a global average pooling layer is applied. This is followed by a fully connected layer. The final output layer produces a probability distribution over the classes, indicating the likelihood of each class being present in the input audio. Fig. 7 shows a simplified layer architecture of the YAMNet model. The depthwise separable convolutional block consists of many sequentially arranged depthwise and pointwise convolutions.

3.5.4. CNN-LSTM

The proposed CNN-LSTM model architecture is tailored for audio classification tasks. The model combines convolutional neu-

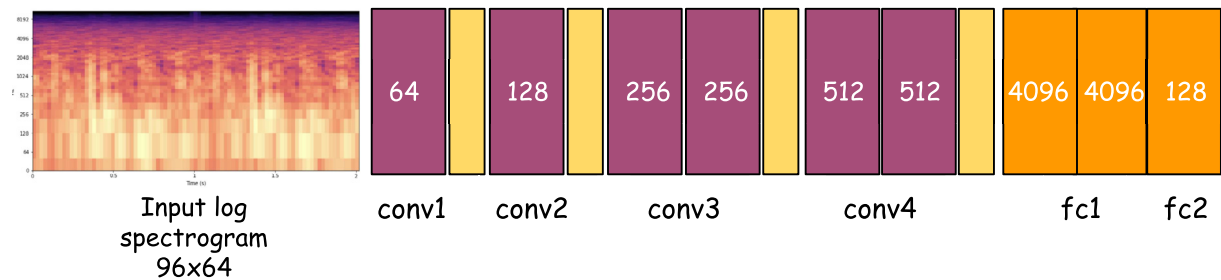


Fig. 6. VGGish model architecture.

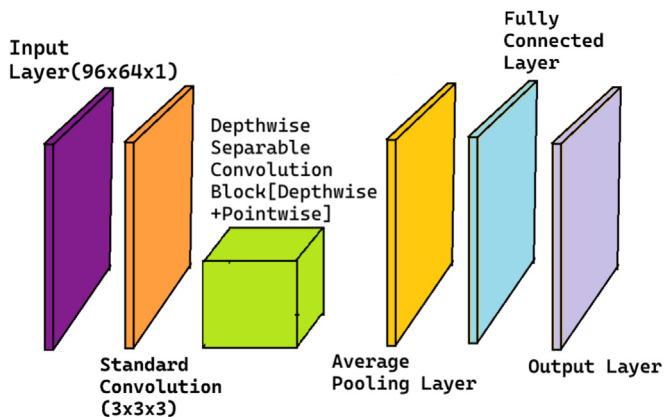


Fig. 7. YAMNet model architecture.

ral networks (CNNs) with long short-term memory (LSTM) networks. It comprises of three distinct branches, each processing different types of audio features: MFCCs (Mel-frequency cepstral coefficients), chroma, and mel spectrograms as shown in the model architecture of our proposed CNN-LSTM model in Fig. 8 and in the model summary in Fig. 9.

The MFCC branch consists of multiple convolutional layers with varying filter sizes, followed by batch normalization and ReLU activation functions. Max-pooling layers are interspersed to reduce spatial dimensions and extract key features. Similarly, the chroma branch employs a similar convolutional structure tailored to chroma fea-

tures. Meanwhile, the mel spectrogram branch processes input spectrograms through a sequence of convolutional and max-pooling layers. The outputs from these branches are concatenated and reshaped to form a unified feature representation, which is then fed into an LSTM layer. The LSTM layer captures temporal dependencies in the audio data, enabling the model to learn long-range dependencies and contextual information crucial for accurate classification. Additional dropout layers are incorporated to mitigate overfitting, followed by fully connected layers for feature aggregation and classification. The final layer utilizes a softmax activation function to predict the probability distribution across different audio classes. The detailed algorithm is defined in Algorithm 1.

Algorithm 1. Simplified CNN-LSTM Algorithm

```
1: procedure CNN_LSTM(input_shapes, num_classes)
2:   1. Initialize Inputs:
3:   ['mfcc', 'croma', 'mSpec']
4:   2. Convolutional Layers:
5:   Apply 2D convolution with 32 filters and strides.
6:   for all filters in [64, 96, 128] do
7:     Conv2D, BatchNormalization, ReLU activation,
       MaxPooling
8:   end for
9:   3. Pooling:
10:  Apply GlobalMaxPooling.
11:  4. Concatenation:
```

(continued on next page)

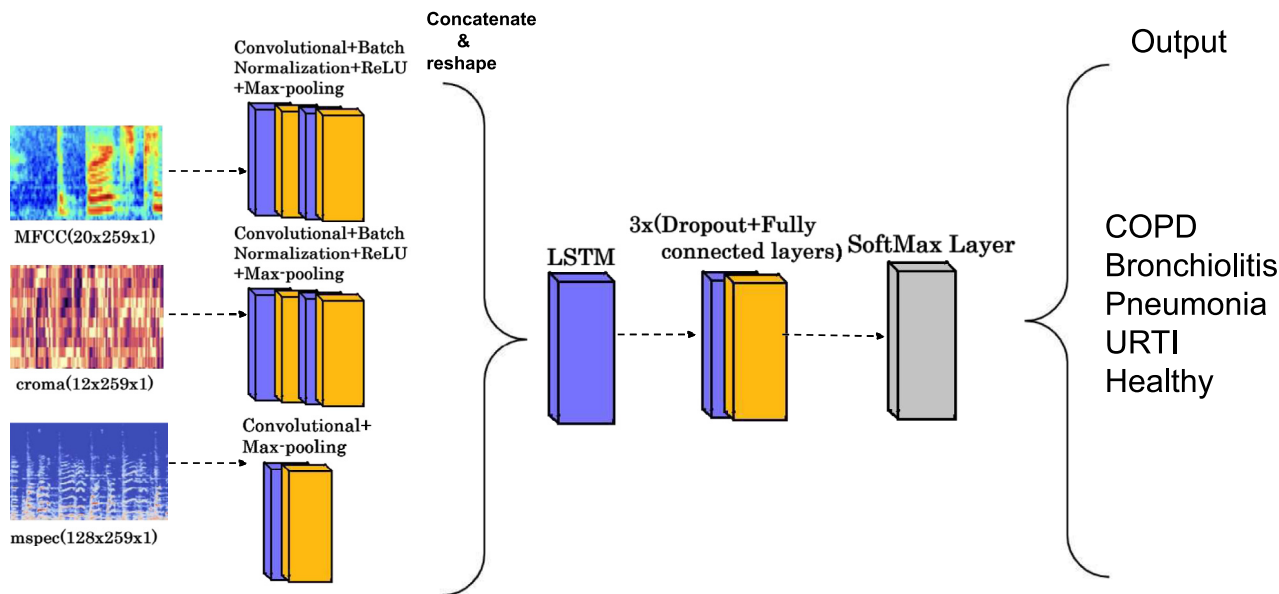


Fig. 8. CNN-LSTM model architecture.

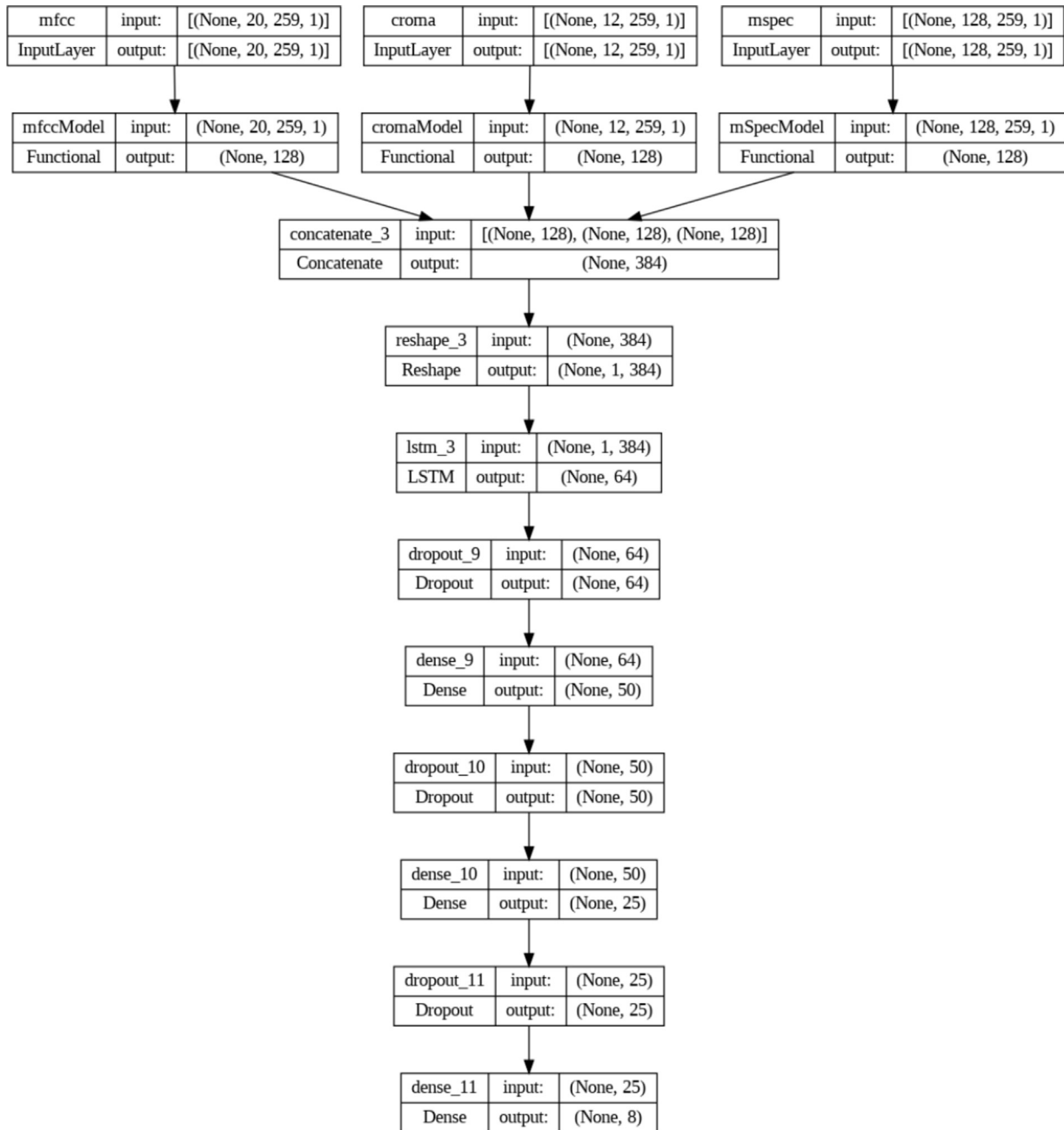


Fig. 9. CNN-LSTM model summary.

```

12: Combine the output vectors from all input types into a
    single vector.
13: 5. LSTM Layer:
14: Apply LSTM layer with 64 units
15: 6. Dense Layers with Dropout:
16: for allunits, rate in [(50, 0.3), (25, 0.3)] do
17:     Pass the LSTM output through dense layers with
        dropout.
18: end for
19: 7. Output Layer:
20: Use softmax layer to produce output probabilities for
    the classes. 21: end procedure
  
```

3.5.5. CNN

The proposed Convolutional Neural Network (CNN) model processes three types of audio features: MFCC (Mel-frequency cepstral coefficients), Chroma STFT, and Mel spectrogram, through separate sub-networks designed to extract distinct patterns from each feature type. Each sub-network consists of convolutional layers followed by batch normalization, ReLU activation, and max-pooling layers, culminating in a global max pooling layer to produce fixed-length feature vectors. The outputs of these sub-networks are concatenated and passed through fully connected dense layers with ReLU activation and dropout regularization, ultimately feeding into a softmax output layer that predicts probabilities for the classes. The model training utilizes callbacks like EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint to optimize performance and prevent overfitting. The model summary is shown below in Fig. 10.

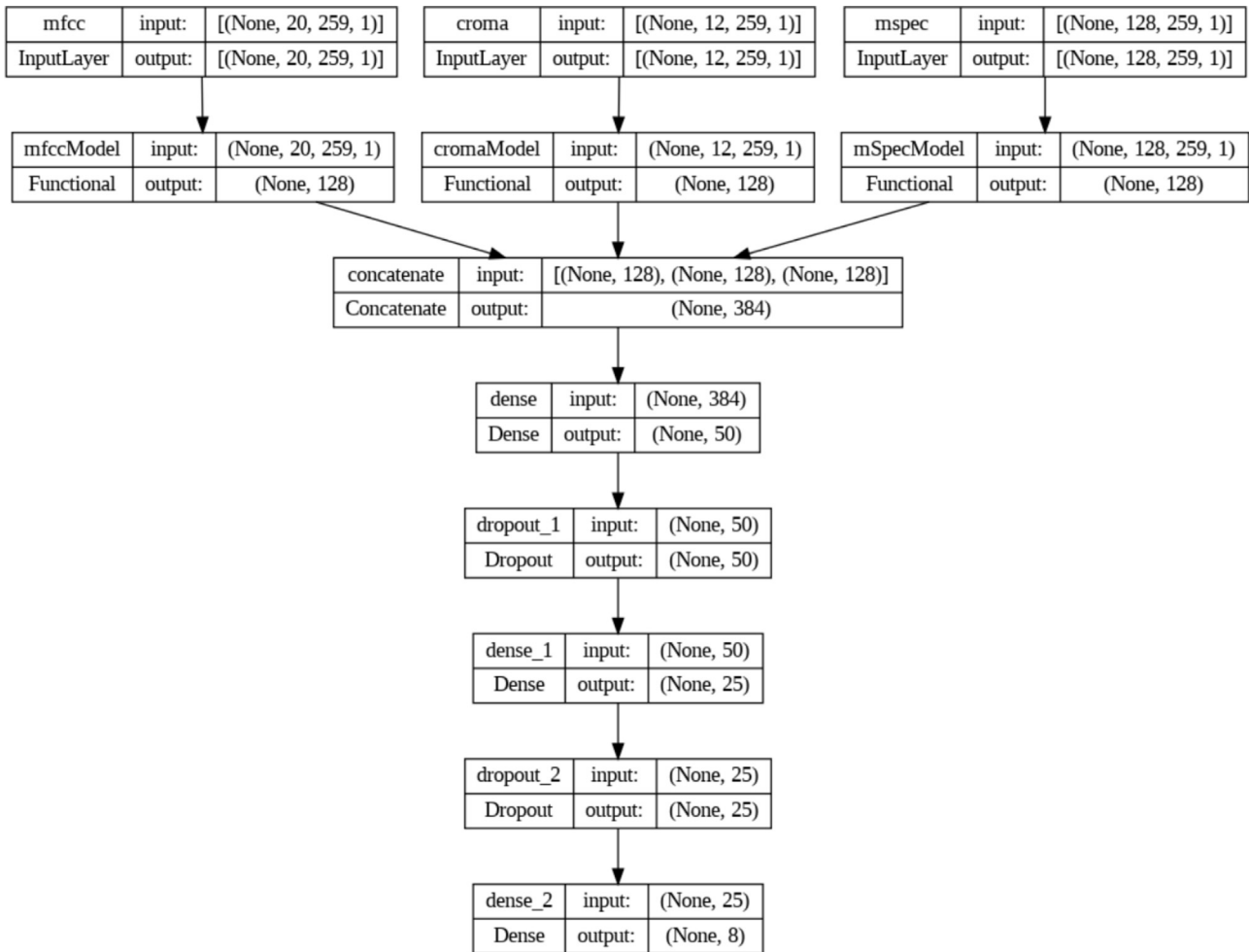


Fig. 10. CNN model summary.

3.6. Quantization

A crucial part of edge deployment is the quantization of TensorFlow models to make them memory-efficient and optimized. Quantization significantly reduces the size of a model and computational requirements, enabling it to run on devices such as smartphones, IoT (Internet of Things) devices, micro-controllers, etc., without an internet connection. According to the analysis in Section 4, the CNN-LSTM model is chosen to be further quantized.

At first, the pruning wrappers used during training are removed using the `tfmot.sparsity.keras.strip_pruning()` function. The pruned model is converted to TFLite format using `tf.lite.TFLiteConverter.from_keras_model()`. Quantization is applied to further optimize the converted TFLite model using `converter.optimizations = [tf.lite.Optimize.DEFAULT]` to enable TensorFlow Lite's default optimization techniques. Finally, the `TFLiteConverter` converts the pruned model into a quantized TFLite model, ready to be deployed on resource-constraint devices.

3.6.1. Pruning

Pruning is a model compression technique that reduces the size and complexity of neural networks by eliminating redundant or less significant weights. In this study, unstructured pruning is applied to our best-performing CNN-LSTM model to enhance its efficiency while preserving its high performance. The pruning process involved removing a portion of the model's weights during training from steps 0 to 1000, guided by a predefined sparsity

schedule. By employing this technique, the size of the original large model (4.34 MB) is significantly reduced to 1.48 MB. Despite this substantial reduction in size, the pruned model maintained a high accuracy of 95%. This highlights the effectiveness of pruning in optimizing model size and computational efficiency without considerable loss in performance.

In this study, we defined a pruning schedule with the following settings: *initial_sparsity* = 0%, *final_sparsity* = 50%, *begin_step* = 0, and *end_step* = 1000. Pruning begins at step 0 and gradually increases to reach the desired sparsity using a polynomial decay function by step 1000. Then, pruning is applied to three submodels individually: *mfcc_model*, *croma_model*, and *mSpec_model*. The outputs of the pruned submodels are concatenated into a larger model, reshaped, and fed as inputs to CNN-LSTM layers. The pruned model is then trained, with `UpdatePruningStep()` added during training to ensure that the pruning step count is updated during each training iteration.

3.6.2. Knowledge distillation

Knowledge distillation³⁶ is a model compression technique that aims to transfer the learnings of a large and computationally expensive model, known as the *teacher model*, to a smaller model called *student model*. The distillation process involves using the probability distribution (soft labels) predicted by the teacher model alongside the one-hot encoded hard labels to train a student model. The student model is trained with a loss function that minimizes the differences between its predictions and both the hard and soft labels.

Table 2
Performance analysis of Quantization. Pruning significantly reduced the model size while maintaining the original and high accuracy of the CNN-LSTM model.

Technique	Accuracy	F1 Score	Size (MB)
Original	96%	96%	4.34
Pruning	95%	95%	1.48
Knowledge Distillation	92%	91%	1.26

Table 3
Size analysis of TFLite quantization. TFLite significantly reduces the model size, making the model more suitable for deployment on edge devices.

Model	Size (MB)	
	Before Pruning	After Pruning
Original	4.34	1.48
TFLite	1.38	1.39
Optimized TFLite	0.71	0.38

In this study, knowledge distillation is applied to our best-performing CNN-LSTM model to reduce its size while maintaining high performance significantly. By utilizing this technique, the size of the large teacher model (4.34 MB) was substantially reduced to 1.26 MB. Despite the reduction in size, the distilled student model achieved an accuracy of 92%. This demonstrates the effectiveness of knowledge distillation in compressing complex models without significant depreciation in performance.

Table 2 presents a performance metrics comparison between the original CNN-LSTM model, model after pruning and that after performing knowledge distillation.

Quantization may often come with a trade-off between accuracy and model size. While the reduction in model size can produce a faster and lighter model, it may also cause a slight degradation of performance. In this study, pruning was applied to reduce the model size from 4.34 MB to 1.48 MB without much compensation of accuracy. While the knowledge distillation produced a much lighter model (1.26 MB), there was a slight loss in the accuracy compared to that of the original model. In our work, we prioritized minimizing accuracy loss while achieving significant reductions in model size, ensuring the model is practical for on-device respiratory disease detection.

3.6.3. TFLite

The process of optimization is carried out by converting the weights and activations from high-precision formats to lower-precision formats. This reduces the model size, which requires less storage and provides faster inference, making it efficient for resource-constraint devices.

TFLite helps optimize the model weights further, making the format deployable to the smartphone application. The pruned CNN-LSTM is chosen for further optimization due to the reasonable trade-off between the metrics of the model before pruning. The quantization results after TFLite conversion can be seen in Table 3.

The table clearly demonstrates the substantial impact of optimization and conversion to TensorFlow Lite (TFLite) on model size reduction. Initially, the model's size was 4.34 MB. Through TFLite conversion, the model size was significantly reduced to 1.38 MB. Further optimization brought the size down to only 0.71 MB. This marked reduction, especially before pruning, highlights the effectiveness of these techniques in making the model more lightweight and suitable for deployment on resource-constrained devices.

3.7. Implementation

The implementation of the system on an edge device, specifically an Android phone, is shown in Fig. 11.

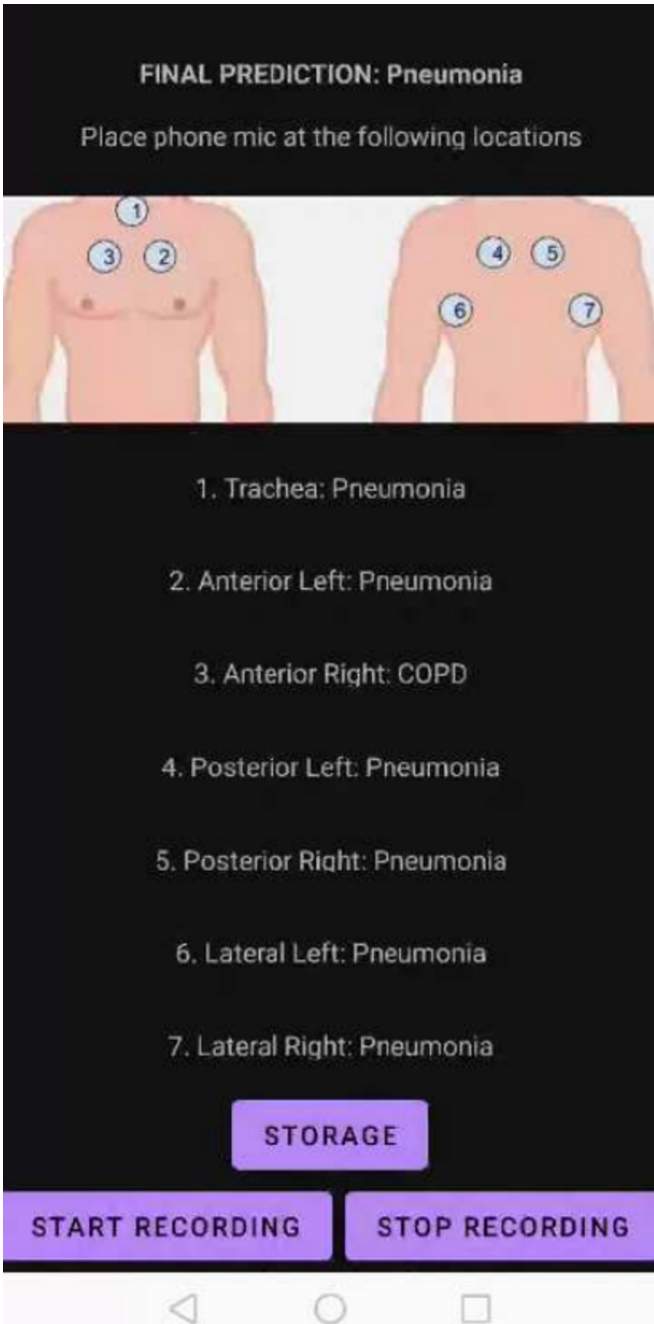


Fig. 11. RespiScan Application.

An Android application, RespiScan, has been developed using Android Studio in Kotlin to facilitate this system. The user can perform lung auscultation using the built-in microphone of their smartphones. This method has been clinically proven effective by H. Ferreira-Cardoso, et al.,³⁷ where a study was conducted with clinicians performing lung auscultation on the smartphone recordings of 134 patients. The clinicians were able to classify accurate diseases by 85%.

The quantized TensorFlow Lite model was deployed for its balance between size and accuracy, making it suitable for deployment on smartphones. The TFLite model was loaded into the application using the TensorFlow Lite Interpreter API.

The application allows the user to record from seven body locations as instructed on the screen (trachea, anterior Right, anterior left, posterior right, posterior left, lateral right, lateral left). Stethoscope recordings can also be inputted by selecting files from the

storage of the user's phone. A stethoscopic device or an audio amplifier can be connected to the mobile phone so that the audio recordings can be noise-free as much as possible. The audio input goes through preprocessing steps such as audio normalization, pre-emphasis, and windowing.

Once the audio data is preprocessed, it is fed into the TFLite model for inference. The model predicts the probability of different respiratory conditions (COPD, URTI, Bronchiolitis, Pneumonia, or Healthy). The app is designed to display the predictions of respiratory disease for each of the seven listed locations. The final conclusion of the predictions is displayed at the top, which is the majority of the seven locations.

4. Results and evaluation

The performance of various deep learning models (CNN-GRU, VGGish, YAMNet, CNN-LSTM, CNN) are evaluated using the metrics accuracy, precision, recall, and F1 score. The formulas used for these metrics are as follows:

4.1. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

4.2. Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

4.3. Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

4.4. F1 Score

$$\text{F1Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where *TP* is True Positives, *TN* is True Negatives, *FP* is False Positives, and *FN* is False Negatives.

The analysis of the performance metrics is shown in Table 4 and the best metrics are highlighted in bold.

In this paper, we explored the effectiveness of using various deep learning models for the detection of respiratory anomalies using deep edge intelligence. The proposed hybrid CNN-LSTM model achieves the highest performance, across all metrics, with an accuracy of 0.96, precision of 0.95, recall of 0.96, and an F1 score of 0.96. This makes CNN-LSTM the most effective model for detecting respiratory anomalies in this study.

Table 4
Performance analysis of the models on the modified ICBHI dataset.

Model	Accuracy	Precision	Recall	F1 Score
CNN-GRU	0.86	0.86	0.85	0.85
VGGish	0.92	0.93	0.92	0.92
YAMNet	0.89	0.83	0.89	0.86
CNN-LSTM	0.96	0.95	0.96	0.96
Pruned CNN-LSTM	0.95	0.95	0.95	0.95
CNN	0.92	0.90	0.92	0.91

Table 5
Parameters and size analysis of the models.

Model	Parameters (M)	Model Size (mb)
CNN-GRU	1.4	16.2
VGGish	9.2	35.0
Yamnet	2.3	8.88
CNN-LSTM	1.38	4.34
CNN	1.0	3.20

Table 6
Inference time analysis of TFLite quantization and pruning.

Model	Time (ms)	
	Before Pruning	After Pruning
Original	287.5	186.2
TFLite	138.8	101.4
Optimized TFLite	68.8	55.4

The VGGish and standard CNN models also performed well, both achieving an accuracy of 0.92. VGGish had a slightly higher precision of 0.93 compared to CNN's 0.92. Both models are closely matched in recall and F1 score as well. YAMNet showed robust recall performance at 0.89, indicating its strength in identifying true positive cases, though its precision was lower at 0.83, resulting in a balanced F1 score of 0.86. The CNN-GRU model, while demonstrating the lowest performance among the evaluated models with an accuracy of 0.86, still maintains a balanced performance across all metrics.

Further, the models are also compared with their parameters and model size, as shown in Table 5. The most lightweight model is highlighted in bold.

Here, CNN-GRU and VGGish architectures are the largest models while CNN-LSTM and CNN architectures are significantly smaller, whilst maintaining a good performance.

It is observed that the best performing model is CNN-LSTM and the least sized model is the basic CNN architecture. However, the size of CNN-LSTM is not drastically larger than the CNN model as it is also quite lightweight. Then, as per the quantization process in Section 3.6, the pruned model, which is of 0.38 MB size, is deployed to the smartphone device. Table 6 shows the resulting inference time of the model before and after pruning, as well as at each stage of optimization to TFLite. It is observed the after pruning, the optimized TFLite, which is also the smallest in size, has the lowest inference

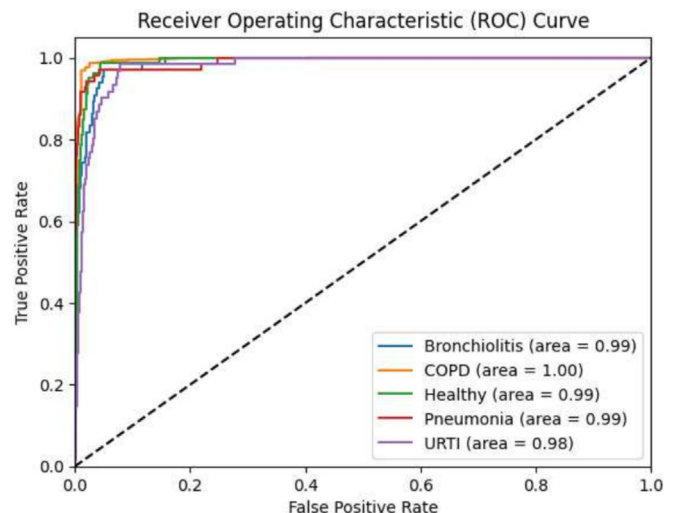


Fig. 12. CNN-LSTM ROC curve.

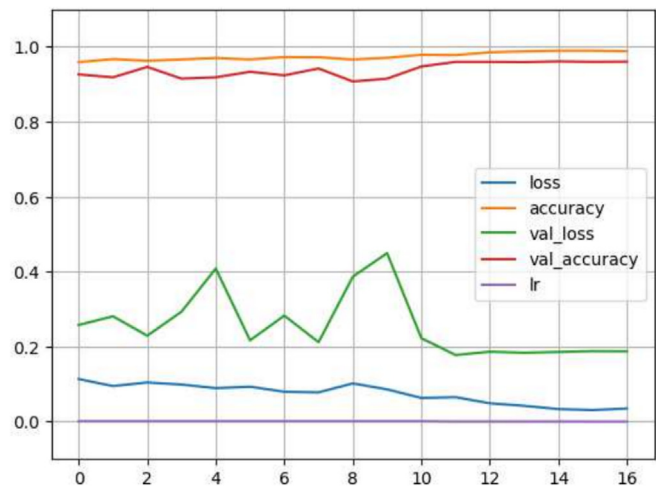


Fig. 13. CNN-LSTM accuracy loss curve.

time. This significantly impacts usage in resource-constraint edge devices, allowing a faster real-time analysis.

Fig. 12 shows the ROC curve. For the ROC curve probabilities for each class using the trained model were calculated. These probabilities were then binarized for each class to generate the ROC curves. This ROC curve results suggest varying levels of discrimination performance across the five classes, where probabilities of 0.99, 1.00, 0.99, 0.99 and 0.98 indicate high confidence in classification with minimal false positives at high true positive rates.

The accuracy and loss curves in Fig. 13 provide valuable insights into the performance of the model during training and validation. The high accuracy and low loss values observed for both training and validation data indicate that the model effectively learns from the training data and generalizes well to unseen validation data.

The confusion matrix in Fig. 14 visually represents the performance of the classification model by comparing the predicted disease labels with the actual labels. In this matrix, the rows correspond to the true disease classes, while the columns represent the predicted disease classes. Each cell in the matrix shows the percentage of instances for a true-predicted class pair, normalized by the total number of instances in the true class.

The diagonal cells indicate the proportion of correctly classified instances for each disease. High values along this diagonal suggest that the model performs well in correctly identifying those diseases. As shown in Fig. 14, the classes bronchiolitis, COPD, healthy predicted performed well and URTI had the lowest performance.

The true positive rate for URTI is 67.21%, which is significantly lower compared to other classes like COPD (98.12%) or Pneumonia (91.55%). This indicates the model struggles more with accurately identifying URTI cases. A large portion of URTI samples is misclassified as Bronchiolitis (13.11%) and Healthy (18.03%). This suggests that URTI most likely has overlapping features with these classes, leading to misclassification.

To test for the generalizability and robustness of the pretrained and custom models, we trained them on another dataset by Fraiwan, M.³⁸ with different recording conditions, patient demographics, and respiratory disease classes. The dataset includes 336 respiratory sound recordings from 112 Middle Eastern patients with different respiratory conditions. It consists of 8 classes of res-

Table 7
Performance analysis of the models on dataset by Fraiwan, M. 38.

Model	Accuracy	Precision	Recall	F1 Score
CNN-GRU	0.49	0.48	0.47	0.47
VGGish	0.54	0.63	0.54	0.58
YAMNet	0.36	0.33	0.35	0.34
CNN-LSTM	0.62	0.61	0.62	0.62
CNN	0.58	0.53	0.56	0.55

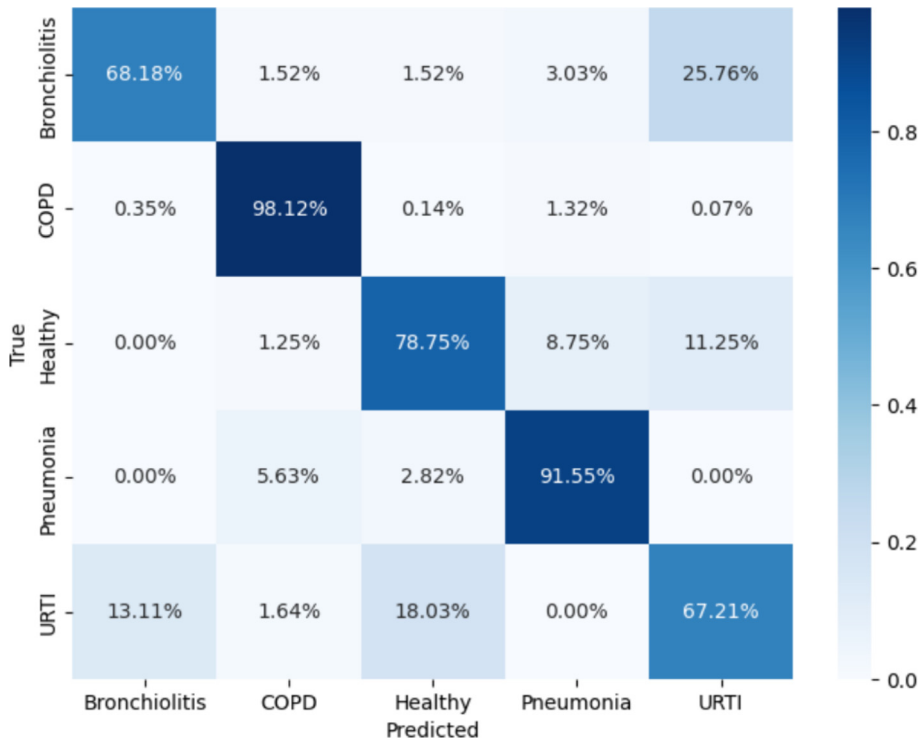


Fig. 14. Confusion matrix.

Table 8
Comparison analysis between prediction models.

Ref	Year	Model	Metrics
12	2021	CNN-based	Accuracy = 93%
14	2021	VGGish-BiGRU	Precision = 95%, Recall = 94%, F1- score = 94%
15	2020	GRU-based	Accuracy = 95.67%±0.77%
16	2021	CRNN	sensitivity = 0.64, specificity = 0.83, average score = 0.74, harmonic score = 0.72
17	2021	ARSC-Net	sensitivity = 46.38%, specificity = 67.13%, scores = 56.76%
18	2021	CNN-MOE	sensitivity = 0.98, specificity = 0.86, score = 0.92
This work	2024	CNN-LSTM	Accuracy = 96%, Precision = 95%, Recall = 96%, F1 Score = 96%

piratory health conditions: Normal, Asthma, Pneumonia, COPD, BRON (Bronchitis), Heart failure, Lung fibrosis, and Pleural effusion. Table 7 presents the performance of all the models trained on the dataset.

Compared to the performance metrics achieved on the ICBHI dataset, the performance of the models drops significantly when trained on a different dataset. This decline could be due to several factors, such as class imbalance and limited data availability. Classes like Pneumonia, BRON, Lung Fibrosis, and Pleural Effusion were notably underrepresented, each comprising fewer than five subjects. The model performance could be improved through careful fine-tuning and by addressing the issue of class imbalance.

Table 8 compares this work with other existing models trained on the ICBHI dataset. The CNN-LSTM model of this work is shown to be the best-performing one for predicting pulmonary diseases from the ICBHI dataset.

In comparison to the work of Naoki Asatoni et al.,¹⁶ which employed an improved CRNN for respiratory sound classification, our CNN-LSTM model achieves significantly higher performance in terms of sensitivity and specificity. Our model achieved sensitivity and specificity values of 0.96 and 0.87, respectively, whereas their CRNN model achieved 0.64 and 0.83, respectively.

Our model demonstrates a substantially higher sensitivity. This means it correctly identifies a greater proportion of true positive cases. This is crucial particularly in medical applications, where it is essential to minimize false negatives and ensure that patients are correctly diagnosed. While both the models achieve high specificity, our model performs better in identifying true negative cases. The higher specificity suggests that our model is better at distinguishing respiratory conditions, thus minimizing false positives.

When evaluating the trade-off between sensitivity and specificity, our model does not sacrifice one for the other. Instead, it achieves high values in both metrics. In medical diagnostics, it is important to maximize sensitivity while also maintaining a high level of specificity. Our model has a satisfactory balance of both, achieving performance metrics that surpass those of the existing work of Naomi Asatoni et al.

5. Conclusion

In this paper, several deep learning models were trained upon the pre-processed ICBHI dataset to detect respiratory diseases (COPD, Bronchiolitis, Pneumonia, URTI, Healthy). The models were evaluated, from which the best-performing model was chosen, further quantized, and then deployed to the smartphone application 'RespiScan'. This application takes audio inputs from the instructed seven locations of the chest and sends them to the pruned and quantized deep learning model, CNN-LSTM, to predict diseases.

For future work, the method can be experimented with real patients in clinical settings to further assess its performance. The ICBHI dataset contains class imbalance issues which is a major challenge in training deep learning models. Over 51% of the samples belong to the COPD class, which skews the model's predictions and introduces bias. It can be improved by incorporating clinical audio data of more common respiratory diseases, particularly asthma and lower respiratory tract infections (LRTI). In addition, including conditions like Pleural Effusion and Atelectasis—commonly associated with various lung conditions—would further enrich the dataset. Expanding the sample size across all classes is also essential to increase the generalizability and robustness of the models. Audio input devices, such as those that support similar quality as that of a stethoscope, can be utilized as an attachment to the smartphone or other edge devices like Raspberry Pi. Additionally, the system can be implemented for Smart Home Health or wearables that continuously monitor respiratory sounds in real-time. Other than the proposed models, extensive deep learning architectures can be evaluated to improve the accuracy rate. Using Explainable AI could also increase the interpretability of the models. This research has the potential to explore healthcare using edge intelligence for respiratory disease diagnosis, especially in remote or underserved areas where access to healthcare professionals is limited.

CRediT authorship contribution statement

Tahiya Tasneem Oishee: Conceptualization, Methodology, Data curation, Software, Visualization, Writing – original draft. **Jareen Anjom:** Methodology, Data curation, Software, Writing – original draft. **Uzma Mohammed:** Methodology, Visualization, Data curation, Writing – original draft. **Md. Ishan Arefin Hossain:** Project administration, Supervision, Writing – review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Forum of International Respiratory Societies. The global impact of respiratory disease. 3rd ed.; 2021.
2. Kjelle E, Brandsaeter IØ, Andersen ER, et al. Cost of low-value imaging worldwide: a systematic review. *Appl Health Econ Health Policy*. 2024;22:485–501. <https://doi.org/10.1007/s40258-024-00876-2>.
3. Tariq Z, Shah SK, Lee Y. Feature-based fusion using CNN for lung and heart sound classification. *Sensors*. 2022;22(4):1521. <https://doi.org/10.3390/s22041521>.
4. Brunese L, Mercaldo F, Reginelli A, Santone A. A neural network-based method for respiratory sound analysis and lung disease detection. *Appl Sci*. 2022;12(8):3877. <https://doi.org/10.3390/app12083877>.
5. Chamberlain DB, Kodgule R, Fletcher RR. A mobile platform for automated screening of asthma and chronic obstructive pulmonary disease. In: *2016 38th Annual international conference of the IEEE engineering in medicine and biology society (EMBC)*. IEEE; 2016:5192–5195. <https://doi.org/10.1109/EMBC.2016.7591897>, August.
6. Pillai AS. Utilizing deep learning in medical image analysis for enhanced diagnostic accuracy and patient care: challenges, opportunities, and ethical implications. *J Deep Learn Genom Data Anal*. 2021;1(1):1–17.
7. Adelaja O, Alkattan H. Operating artificial intelligence to assist physicians diagnose medical images: a narrative review. *Mesopotamian J Artif Intell Healthcare*. 2023;2023:45–51. <https://doi.org/10.58496/MJAIH/2023/009>.
8. Bohadana A, Izbicki G, Kraman SS. Fundamentals of lung auscultation. *New England J Med*. 2014;370(8):744–751. <https://doi.org/10.1056/NEJMra1302901>.

9. Reichert S, Gass R, Brandt C, Andr  s E. Analysis of respiratory sounds: state of the art. *Clin Med Circulat, Respir Pulmon Med* 2008;2:CCRPM-S530.<https://doi.org/10.4137/CCRPM.S530>.
10. Flietstra B, Markuzon N, Vyshedskiy A, Murphy R. Automated analysis of crackles in patients with interstitial pulmonary fibrosis. *Pulmon Med.* 2011;2011(1):590506. <https://doi.org/10.1155/2011/590506>.
11. Pramono RXA, Bowyer S, Rodriguez-Villegas E. Automatic adventitious respiratory sound analysis: a systematic review. *PLoS One.* 2017;12(5):e0177926. <https://doi.org/10.1371/journal.pone.0177926>.
12. Srivastava A, Jain S, Miranda R, Patil S, Pandya S, Kotecha K. Deep learning based respiratory sound analysis for detection of chronic obstructive pulmonary disease. *PeerJ Comput Sci.* 2021;7:e369. <https://doi.org/10.7717/peerj-cs.369>.
13. Kim Y, Hyon Y, Jung SS, Lee S, Yoo G, Chung C, Ha TY. Respiratory sound classification for crackles, wheezes, and rhonchi in the clinical field using deep learning. *Scient Rep.* 2021;11(1). <https://doi.org/10.1038/s41598-021-96724-7>.
14. Lal KN. A lung sound recognition model to diagnose respiratory diseases by using transfer learning. *Multimedia Tools Appl.* 2023;82(23):36615–36631. <https://doi.org/10.1007/s11042-023-14727-0>.
15. Basu V, Rana S. Respiratory diseases recognition through respiratory sound with the help of deep neural network. In: 2020 4th International conference on computational intelligence and networks (CINE). February 2020:1–6. IEEE. doi: 10.1109/CINE48825.2020.234388.
16. Asatani N, Kamiya T, Mabu S, Kido S. Classification of respiratory sounds by generated image and improved CRNN. In: 2021 21st International Conference on Control, Automation and Systems (ICCAS). October 2021:1804–1808. IEEE. <https://doi.org/10.23919/ICCAS52745.2021.9649906>.
17. Xu L, Cheng J, Liu J, Kuang H, Wu F, Wang J. Arsc-net: Adventitious respiratory sound classification network using parallel paths with channel-spatial attention. In: 2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). December 2021:1125–1130. IEEE. doi: 10.1109/BIBM52615.2021.9669787.
18. Pham L, Phan H, Palaniappan R, Mertins A, McLoughlin I. CNN-MoE based framework for classification of respiratory anomalies and lung disease detection. *IEEE J Biomed Health Inform.* 2021;25(8):2938–2947. <https://doi.org/10.1109/JBHI.2021.3064237>.
19. Fraiwan M, Fraiwan L, Alkhodari M, Hassanin O. Recognition of pulmonary diseases from lung sounds using convolutional neural networks and long short-term memory. *J Ambient Intell Human Comput.* 2021;13(10):4759–4771. <https://doi.org/10.1007/s12652-021-03184-y>.
20. Nathan V, Vatanparvar K, San Chun K, Kuang J. Utilizing deep learning on limited mobile speech recordings for detection of obstructive pulmonary disease. In: 2022 44th Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC). July 2022:1338–1341. IEEE. doi: 10.1109/EMBC48229.2022.9871980.
21. ICBHI 2017 Challenge – ICBHI Challenge. (n.d.). Retrieved June 24, 2022, from https://bchchallenge.med.auth.gr/ICBHI_2017_Challenge
22. Abdul ZK, Al-Talabani AK. Mel frequency cepstral coefficient and its applications: a review. *IEEE Access.* 2022;10:122136–122158. <https://doi.org/10.1109/ACCESS.2022.3223444>.
23. Zhao X, Wang D. Analyzing noise robustness of MFCC and GFCC features in speaker identification. In 2013 IEEE International Conference on Acoustics, Speech and Signal Processing. May 2013:7204–7208. IEEE. doi: 10.1109/ICASSP.2013.6639061.
24. Rao KS, Manjunath KE. *Speech recognition using articulatory and excitation source features*. Springer; 2017. <https://doi.org/10.1007/978-3-319-49220-9>.
25. Feldman HA, Kaiser N, Peacock JA. Power spectrum analysis of three-dimensional redshift surveys. *arXiv preprint.* 1993. <https://doi.org/10.48550/arXiv.astro-ph/9304022>.
26. Yin H, Hohmann V, Nadeu C. Acoustic features for speech recognition based on Gammatone filterbank and instantaneous frequency. *Speech Commun.* 2011;53(5):707–715. <https://doi.org/10.1016/j.specom.2010.04.008>.
27. Strang G. The discrete cosine transform. *SIAM Rev.* 1999;41(1):135–147. <https://doi.org/10.1137/S0036144598336745>.
28. Ustubioglu A, Ustubioglu B, Ulutas G. Mel spectrogram-based audio forgery detection using CNN. *Signal, Image Video Process.* 2023;17:2211–2219. <https://doi.org/10.1007/s11760-022-02436-4>.
29. M  ller M. *Short-time Fourier transform and chroma features*. Lab Course: Friedrich-Alexander-Universit  t Erlangen-N  rnberg; 2015.
30. Dey R, Salem FM. Gate-variants of gated recurrent unit (GRU) neural networks. In: 2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS). August 2017:1597–1600. IEEE. doi: 10.1109/MWSCAS.2017.8053243.
31. Lian Z, Li Y, Tao J, Huang J. Investigation of multimodal features, classifiers and fusion methods for emotion recognition. *arXiv preprint.* 2018. <https://doi.org/10.48550/arXiv.1809.06225>.
32. Hershey S, Chaudhuri S, Ellis DP, Gemmeke JF, Jansen A, Moore RC, Wilson K. CNN architectures for large-scale audio classification. In: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). March 2017:131–135. IEEE. doi: 10.1109/ICASSP.2017.7952132.
33. Gemmeke JF, Ellis DP, Freedman D, Jansen A, Lawrence W, Moore RC, Ritter M. Audio set: An ontology and human-labeled dataset for audio events. In: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). March 2017:776–780. IEEE. doi: 10.1109/ICASSP.2017.7952261.
34. Mahum R, Irtaza A, Javed A. EDL-Det: A robust TTS synthesis detector using VGG19-based YAMNet and ensemble learning block. *IEEE Access.* 2023;11:134701–134716. <https://doi.org/10.1109/ACCESS.2023.3332561>.
35. Haase D, Amthor M. Rethinking depthwise separable convolutions: How intra-kernel correlations lead to improved MobileNets. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2020:14600–14609. *arXiv preprint.* <https://doi.org/10.48550/arXiv.2003.13549>.
36. Alkhulaifi A, Alsahli F, Ahmad I. Knowledge distillation in deep learning and its applications. *PeerJ Comput Sci.* 2021;7:e474. <https://doi.org/10.7717/peerj-cs.474>.
37. Ferreira-Cardoso H, J  come C, Silva S, Amorim A, Redondo M, Fontoura-Matias M, Vicente-Ferreira M, Vieira-Marques P, Valente J, Almeida R, Fonseca J, Azevedo I. Lung auscultation using the smartphone—feasibility study in real-world clinical practice. *Sensors.* 2021;21(14):4931. <https://doi.org/10.3390/s21144931>.
38. Fraiwan M, Fraiwan L, Khassawneh B, Ibnian A. A dataset of lung sounds recorded from the chest wall using an electronic stethoscope. *Data in Brief.* 2021;35:106913. <https://doi.org/10.17632/jwyy9np4gv.3>.